What You'll Get From This

A working folder system that tells Claude where it is, what to do, and where to put the work. Three layers. Plain English. No code, no frameworks, no agents.

Video: Folder Architecture (23 Min)

If you want the written version first, keep reading. If you learn better by watching, start the video above and come back here for reference. Both cover the same ground.

The Lesson

The problem with how most people use AI

Right now, most people open Claude or ChatGPT, type something, get a response, and start over. Maybe they save a good prompt somewhere and paste it in again next time. Maybe they hit the token limit and have to start a new conversation. Maybe they spend half the message just re-explaining who they are and what they are working on.

That works for quick questions. It falls apart the moment you try to do real, ongoing work. You are burning tokens on stuff that does not matter. You cannot edit what the AI produces at each step. And every conversation starts from zero.

The folder structure fixes all of that.

What tokens are and why this matters

Before we get into the structure, you need to understand one thing: tokens.

A token is roughly three quarters of a word. Sometimes a single word. Sometimes a long word like "hamburger" is three tokens. The term comes from NLP research in the 1990s. Researchers needed a unit smaller than a word because language does not break the same way across every language. They borrowed "token" from linguistics, which borrowed it from old English

"taken," meaning a sign or a symbol. A token is just the smallest meaningful chunk of a sentence.

There are only so many tokens an AI can hold in its context window before it starts failing. When people say "context window," they mean how many tokens the AI can see at once. That window is finite.

So if you dump everything into one conversation, an AI writing a blog post is also reading your video production notes. You are wasting tokens on information that has nothing to do with the task. The folder structure solves this by separating your work into areas and only loading what is needed for the task at hand.

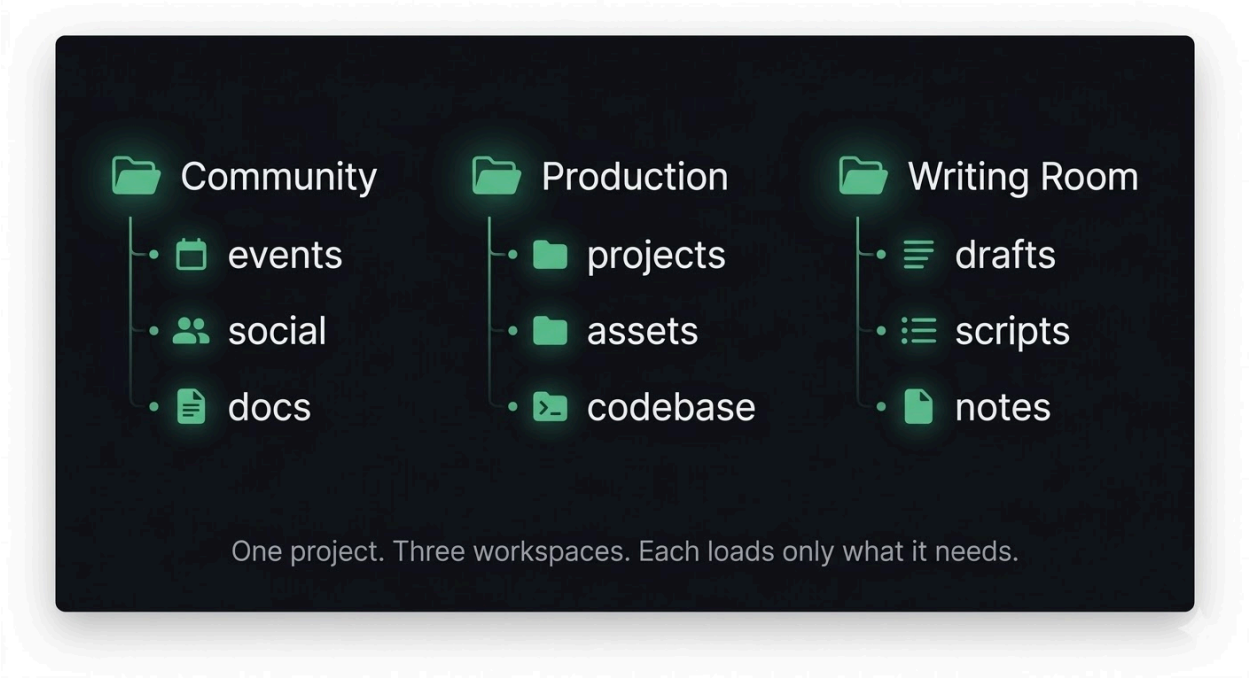
The workspace blueprint: three workspaces for three kinds of work

Here is the concept. Instead of one giant conversation or one massive file, you break your work into separate workspaces. Each workspace handles a different kind of work.

For example:

- **Community** — content, docs, community management
- **Production** — scripts, animations, code, builds
- **Writing Room** — blog posts, newsletters, thinking, client work

These are just examples. Your workspaces will look different depending on what you do. A freelancer might have Client Intake, Delivery, and Admin. A developer might have Frontend, Backend, and Docs. The names change. The structure stays the same.



What VS Code is (and why you do not need it)

In the video, you will see me working inside VS Code. VS Code is an IDE, a developer environment that lets you see all your folders and files in a sidebar and open them without double-clicking through your file system. It makes it easier to bounce between files.

You do not need VS Code to do any of this. You can work inside your regular file explorer, create folders, create text files, and edit them in Notepad or any text editor. You can also use Claude Cowork, which operates inside folders the same way. VS Code is just cleaner for managing lots of files. If it feels overwhelming, skip it for now and use whatever you are comfortable with.

What markdown is and why we use it

Every file in this system is a markdown file (.md). If you have never seen one before, markdown is just a text file with some lightweight formatting.

- Dashes make bullets
- Hashtags (pound signs) make headers
- Asterisks make **bold** or *italic*

That is it. When you talk to Claude and see its responses with bolding, headers, and clean formatting, it is already writing in markdown. Copy a Claude response and paste it into a plain text file. All the formatting turns into markdown symbols. That is how it was making it look that way.

A man named John Gruber created markdown in 2004. The idea was: write something readable as plain text that can also render into a formatted document. He named it "markdown" as a play on "markup language" (like HTML). Markdown strips away all the tags and complexity and makes it look like plain English with a few symbols.

Claude reads markdown cleanly. That is why we use it for every file in the system.

The three layer routing system

This is the core of the whole thing. Three layers. Each one has a job.

Layer 1: The Map (CLAUDE.md)

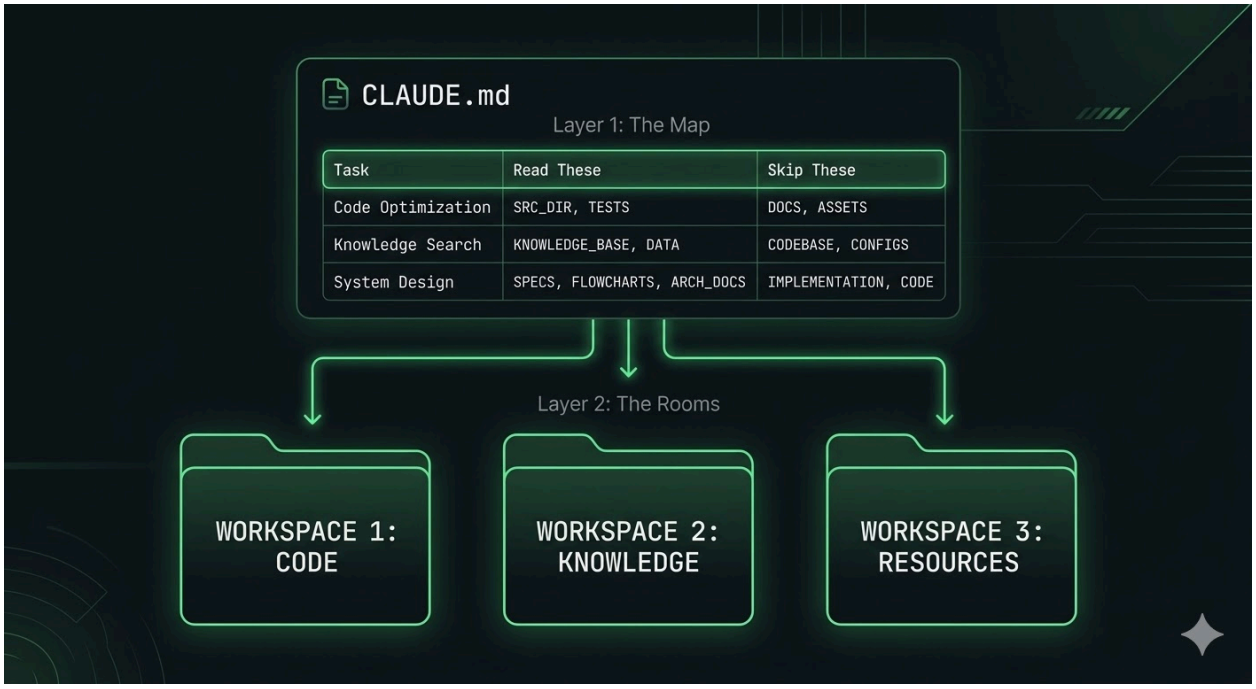
This is the top-level file. It sits at the root of your project folder. Every time Claude enters this workspace, it reads this file first.

Think of it as the floor plan. You walk into any building, the floor plan is on the wall, and you know where to go. The CLAUDE.md file tells the AI:

- What this project is
- What the folder structure looks like
- Naming conventions for files
- Where things go

This is the most important pattern in the whole system. Inside it, you put a simple table that tells the AI: for this task, read these files, skip those files, you might need these skills.

Without this, the AI either reads everything and wastes tokens, guesses wrong about what matters, or produces work you cannot edit along the way. This table eliminates all of those problems.



Layer 2: The Rooms (Workspace Context Files)

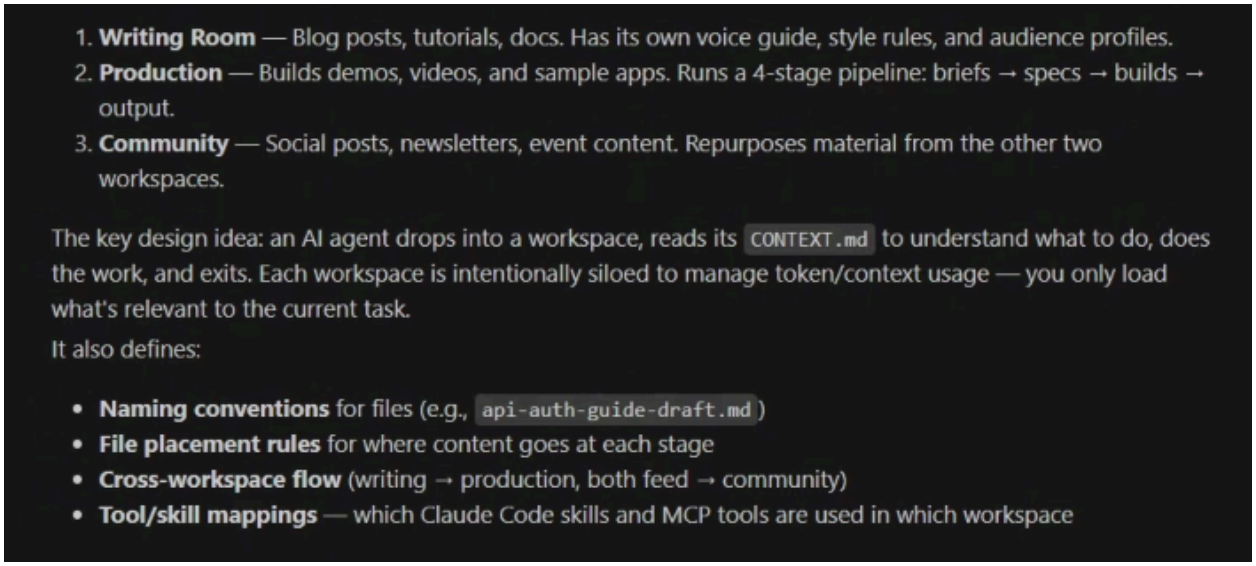
Each workspace has its own context file. When you tell Claude to work in the Writing Room, it reads the Writing Room context file. When you are in Production, it reads the Production context file. It only loads what it needs for where it is.

These context files describe:

- What this workspace is for
- What the process looks like (first I do this, then I do that)
- What files are in here and how they are organized
- What skills or tools to use in this workspace

You can write these by hand or have Claude help you write them. They are plain English. Short documents. A few paragraphs.

In the video, you can see what happens with almost no prompting at all. You say "go to writing room, let's start making something" and the AI immediately reads the context file, understands the workspace, loads the right voice and style, and asks what you want to build. One prompt. That is the power of Layer 2.



Layer 3: The Tools (Skills, MCP Servers, and Plug-and-Play)

Layer 3 is where skills and tools live. Skills are processes that someone figured out and packaged into a set of files that tell Claude how to do a specific thing. A PowerPoint skill. A humanizer skill. A doc co-authoring skill.

The key here: you do not load every skill into every workspace. You wire skills into the workspaces where they are needed. Your Production workspace might reference a front-end design skill and a web app testing skill. Your Writing Room might reference a humanizer skill and a doc co-authoring skill.

You can reference 15, 20, or 100 skills in a project, but each workspace only loads the ones it needs. That is the plug-and-play idea. And if you need Claude to find a new skill or create one, you can wire that ability in too.

Naming conventions that replace databases

One more thing that makes this system work without any code or databases. In your CLAUDE.md, you add naming conventions.

If a blog draft is created, it gets named like: `api-auth-guide_draft.md` If it is a newsletter, it gets named like: `2026-03-launch-week.md` If it is a demo script version 2, it gets named like: `demo_v2.md`

The AI knows how to find, organize, and move files because the naming tells it everything. You can say "pull my demo v2 and build a spec from it" and Claude knows exactly where to look,

what to pull, and what to do next. No SQL. No vector database. No Python injection. Just naming conventions.

How to make this yours

The template in the video uses a fake project with fake blog posts and demo scripts. That is intentional. You swap the names and rewrite the context files for your own work. The layers stay the same.

If you are a content creator:

- Writing Room → Script Lab
- Production → Edit Bay
- Community → Distribution Hub

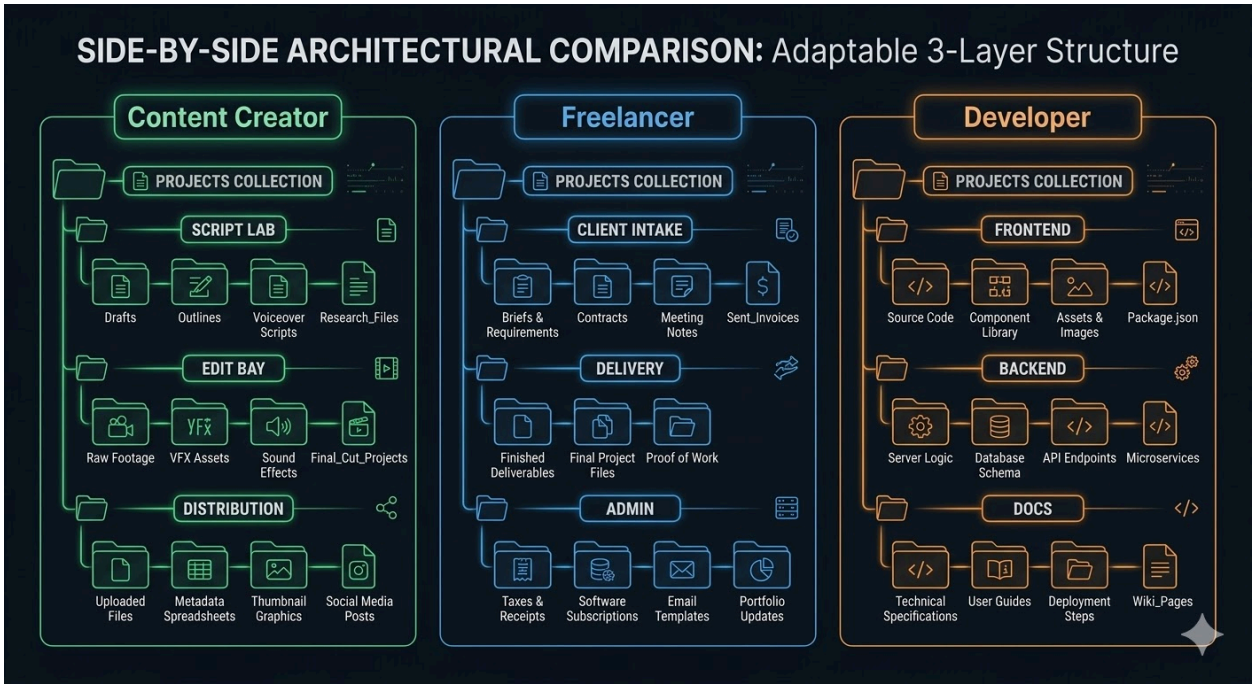
If you are a freelancer:

- Swap workspaces for: Client Intake, Delivery, Admin
- Your context files describe your client process instead of content production

If you are a developer:

- Swap workspaces for: Frontend, Backend, Docs
- Wire in the skills you actually use (testing, deployment, code review)

The three-layer routing system (map → rooms → tools) works the same way no matter what you do. You change the labels and the context. The architecture holds.



The research behind this

This is not a prompt trick. The thinking behind it comes from decades of software engineering, all the way back to 1972. Separation of concerns. Modular composition. The idea that systems work better when each part does one job and the routing between parts is clear.

There is a full research paper on this if you want to go deep. It traces the history from early programming principles through to modern AI workflows and lays out a five-layer architecture (this video covers the three most important layers). You can download it and give it to Claude to explain it to you.

Research Paper: **[ICM PAPER]**

Quick start: your first three files

If you want to start right now with the simplest possible version, here it is. Make a folder. Put three files in it.

CLAUDE.md — Tell Claude who you are, what this project is, and where things go.

```
None
# Identity
You are helping [YOUR NAME] with [WHAT YOU DO].

# Folder Structure
- /drafts – work in progress
- /final – finished outputs
- /references – background material

# Rules
- Read this file first on every new task
- Ask before creating files outside of /drafts
- When unsure, ask
```

CONTEXT.md — Describe the current project.

```
None
# Current Project
[What you are building or working on. 2-3 sentences.]

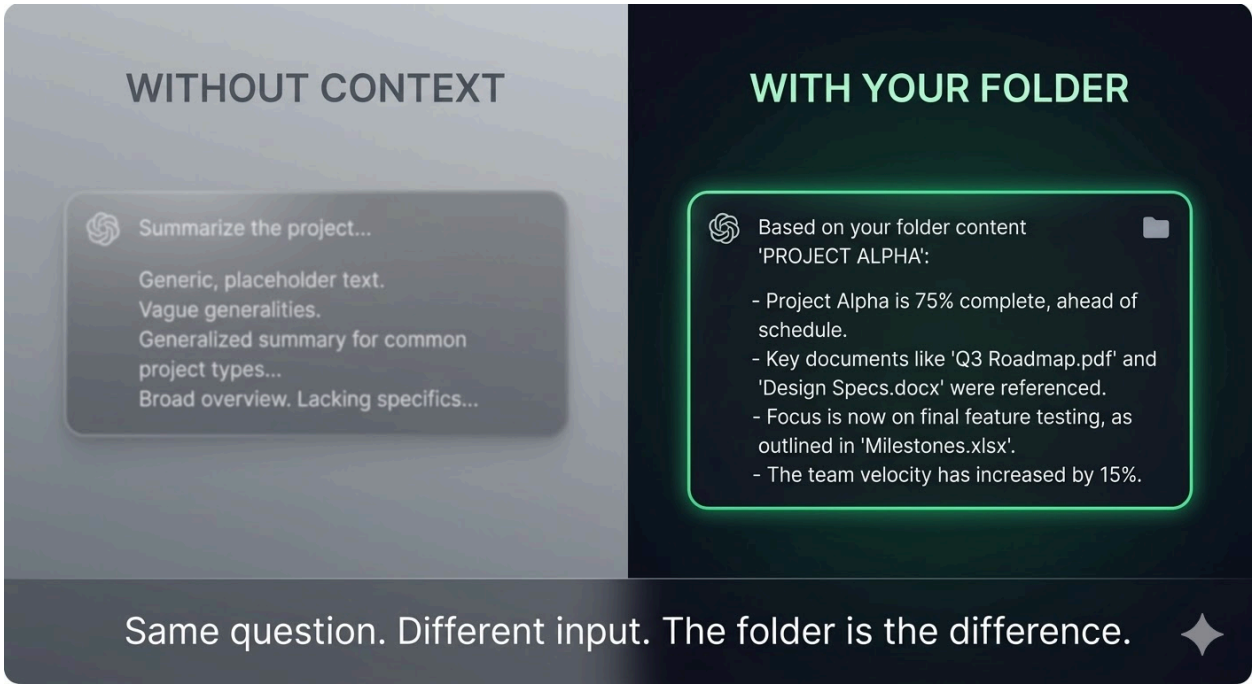
# What good looks like
[What a successful output looks like.]

# What to avoid
[Common mistakes or things you do not want.]
```

REFERENCES.md — Background material Claude should know about.

```
None
# References
[Examples, links, style guides, notes. Anything Claude should have
access to but does not need to act on directly.]
```

Point Claude at the folder (using Claude Code, or upload the files to a Claude Project, or paste the contents into your first message). Ask it something related to your project. Notice the difference.


WITHOUT CONTEXT
Summarize the project...
Generic, placeholder text.
Vague generalities.
Generalized summary for common project types...
Broad overview. Lacking specifics...
WITH YOUR FOLDER
Based on your folder content 'PROJECT ALPHA':
- Project Alpha is 75% complete, ahead of schedule.
- Key documents like 'Q3 Roadmap.pdf' and 'Design Specs.docx' were referenced.
- Focus is now on final feature testing, as outlined in 'Milestones.xlsx'.
- The team velocity has increased by 15%.
Same question. Different input. The folder is the difference.